

# Navegación de costo mínimo sobre terreno real mediante Aprendizaje por Refuerzo Profundo

Integrante 1, Integrante 2, Integrante 3,  
Integrante 4, Integrante 5  
*Maestría en Inteligencia Artificial (MIA-204)*  
*Aprendizaje por Refuerzo*  
Arequipa, Perú

**Resumen**—Un agente aprende a cruzar un terreno real (las faldas del volcán Misti, en Arequipa) gastando el menor esfuerzo por pendiente. En el trabajo parcial usamos Q-learning tabular con una meta fija. Aquí subimos la dificultad y aplicamos Aprendizaje por Refuerzo Profundo. El mapa va a resolución completa ( $108 \times 108$  celdas de 30 m), el agente solo ve un parche  $9 \times 9$  de pendiente a su alrededor más un vector hacia la meta, y el punto de partida cambia en cada episodio. La función de valor la aproxima una red neuronal (Deep Q-Network, DQN). Comparamos dos familias de métodos, los basados en valor (DQN y Double DQN) y los basados en política (REINFORCE), y medimos contra el óptimo exacto de Dijkstra. El agente llega a la meta desde puntos de partida que nunca vio en entrenamiento, con una ruta cercana al óptimo, y sigue funcionando cuando el mundo se vuelve estocástico. Reportamos también dos problemas de diseño y cómo los resolvimos: el *reward shaping* potencial de libro creaba una trampa que impedía aprender, y lo cambiamos por *shaping* de progreso; y con metas totalmente aleatorias los valores  $Q$  divergen, lo que explica por qué elegimos meta fija.

**Index Terms**—Aprendizaje por Refuerzo Profundo, Deep Q-Network, REINFORCE, navegación, reward shaping, modelo digital de elevación.

## I. INTRODUCCIÓN

Cruzar un terreno gastando el menor esfuerzo es un problema viejo. Si conoces el mapa completo, lo resuelves exacto con Dijkstra. Pero un agente autónomo casi nunca tiene el mapa entero: ve su alrededor y decide con información parcial. El Aprendizaje por Refuerzo (Reinforcement Learning, RL) sirve justo para ese caso: aprende una política por prueba y error.

En el trabajo parcial modelamos el terreno como un Proceso de Decisión de Markov (Markov Decision Process, MDP) y lo resolvimos con Q-learning tabular. La malla era reducida ( $54 \times 54$ ), y el inicio y la meta eran fijos. Ese enfoque guarda una tabla de valores y no sirve para situaciones nuevas: memoriza, no generaliza.

En este trabajo subimos la dificultad y usamos Aprendizaje por Refuerzo Profundo (Deep Reinforcement Learning, DRL). Aportamos tres cosas:

- Un entorno más difícil: resolución completa ( $108 \times 108$ ), observación local (un parche  $9 \times 9$  más el vector a la meta) e inicio aleatorio. Esto obliga al agente a generalizar y justifica usar una red neuronal en lugar de una tabla.

- Una comparación de dos familias de RL en el mismo terreno: basada en valor (DQN, Double DQN) y basada en política (REINFORCE), más la línea base exacta de Dijkstra.
- Dos problemas de diseño que encontramos y resolvimos, con datos que lo respaldan: la trampa del *shaping* potencial y la divergencia de  $Q$  con metas aleatorias.

En cuanto al método, modelamos el terreno como un MDP sobre un modelo digital de elevación real, con un costo de paso que crece con la pendiente. El agente observa un parche local y un vector a la meta, y aproxima  $Q(s, a)$  con un perceptrón de dos capas ocultas entrenado por DQN, con reproducción de experiencias y red objetivo. Entrenamos con un *curriculum* que aleja poco a poco el punto de partida, y evaluamos la política greedy sobre 20 inicios que el agente nunca vio, midiendo tasa de éxito y brecha de costo contra la ruta óptima de Dijkstra.

El resto del documento se organiza así. La Sec. II revisa los trabajos previos y ubica nuestro aporte. La Sec. III resume la teoría de los algoritmos que usamos. La Sec. IV define el MDP, la observación y el diseño de la recompensa. La Sec. V detalla la implementación, el pseudocódigo y los hiperparámetros. La Sec. VI presenta los resultados y la Sec. VII los discute. La Sec. VIII cierra con las conclusiones y el trabajo futuro.

## II. TRABAJOS RELACIONADOS

### II-A. Planificación clásica de rutas

El problema de la ruta de costo mínimo sobre una malla está resuelto desde hace décadas. El algoritmo de Dijkstra [1] da el óptimo exacto en tiempo polinomial, y sus variantes con heurística lo aceleran. La condición es fuerte: hace falta el grafo completo y un mundo determinista. Nosotros no competimos con Dijkstra en ese terreno; lo usamos como *línea base exacta* para medir cuánto pierde un agente que solo ve un parche local y que además debe tolerar transiciones estocásticas, dos supuestos que la planificación clásica no cubre.

### II-B. RL tabular sobre grillas

El gridworld es el caso de estudio estándar del Aprendizaje por Refuerzo [8], y Q-learning tabular lo resuelve bien cuando el número de estados es chico. En nuestro trabajo parcial seguimos esa línea: malla  $54 \times 54$ , inicio y meta fijos, una tabla  $Q$  con una entrada por par (celda, acción). El límite aparece

al crecer el problema. Con la malla completa de  $108 \times 108$  y un inicio que cambia en cada episodio, la tabla tendría que almacenar del orden de  $10^8$  entradas y, peor aún, no compartiría nada entre estados parecidos: memoriza rutas en lugar de aprender a navegar. Este trabajo ataca justo ese límite.

### II-C. Métodos profundos basados en valor

DQN [2] reemplaza la tabla por una red neuronal y estabiliza el entrenamiento con dos mecanismos: reproducción de experiencias y una red objetivo congelada. Double DQN [3] corrige después el sesgo optimista que introduce el operador máx. Liu y Zou [4] midieron el peso real de la memoria de reproducción apagándola y variando su tamaño, con juegos de Atari como banco de pruebas. Nosotros repetimos ese estudio de ablación, pero sobre un modelo digital de elevación real en lugar de píxeles de videojuego, y con un espacio de observación de baja dimensión (83 números): un escenario donde no era obvio que los dos mecanismos siguieran siendo indispensables.

### II-D. Métodos basados en política

REINFORCE [6] optimiza la política directamente por gradiente, sin pasar por una función de valor, y es el punto de partida de la familia actor-critic [7]. Las comparaciones publicadas entre ambas familias suelen hacerse sobre entornos de referencia (Atari, MuJoCo). Nuestro aporte aquí es más modesto y más específico: corremos DQN y REINFORCE sobre el mismo terreno, con el mismo conjunto de pares de test y la misma métrica, para mostrar el contraste de estabilidad y varianza en un problema de navegación concreto.

### II-E. Diseño de la recompensa

Ng, Harada y Russell [5] probaron que el *shaping* de forma potencial  $F = \gamma \Phi(s') - \Phi(s)$  no altera la política óptima. El resultado es correcto y se cita como receta segura. Lo que reportamos es un caso práctico donde esa receta falla: con  $\gamma < 1$  y distancias grandes, la garantía teórica sobrevive pero el agente aprendido no, porque el residuo de descuento premia oscilar lejos de la meta (Sec. VII). No conocemos este efecto documentado con datos en un problema de navegación sobre terreno real, y es uno de los aportes del trabajo.

### II-F. Políticas condicionadas a la meta

Cuando la meta cambia entre episodios, el enfoque habitual es condicionar la función de valor a la meta (Universal Value Function Approximators, UVFA) [9], reetiquetar las trayectorias fallidas (Hindsight Experience Replay, HER) [11] o priorizar el muestreo del búfer (Prioritized Experience Replay, PER) [10]. Nosotros no aplicamos estas técnicas; medimos el problema que las motiva. Mostramos que con metas aleatorias y DQN estándar los valores  $Q$  divergen y la tasa de éxito no se sostiene, lo que justifica nuestra decisión de alcance (meta fija) y define la línea de trabajo futuro.

### II-G. Resumen del aporte

Frente a lo anterior, este trabajo aporta tres cosas: un entorno de navegación sobre terreno real con observación local e inicio aleatorio donde la tabla deja de ser viable; una comparación controlada de las familias basada en valor y basada en política contra el óptimo exacto de Dijkstra; y dos hallazgos negativos documentados con datos, la trampa del *shaping* potencial y la divergencia de  $Q$  con metas aleatorias.

## III. MARCO TEÓRICO

### III-A. MDP y $Q$ -learning

Un MDP se define por estados  $s$ , acciones  $a$ , una transición  $P(s'|s, a)$  y una recompensa  $r(s, a)$ . El objetivo es una política  $\pi$  que maximice el retorno descontado  $\sum_t \gamma^t r_t$ .  $Q$ -learning aprende la función de valor de acción  $Q(s, a)$  con la actualización por diferencia temporal (Temporal Difference, TD):

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]. \quad (1)$$

La política greedy es  $\pi(s) = \arg \max_a Q(s, a)$ .

### III-B. Deep $Q$ -Network (DQN)

Cuando hay muchos estados, la tabla  $Q$  no cabe. DQN [2] la reemplaza por una red neuronal  $Q(s, a; \theta)$  y minimiza la pérdida

$$\mathcal{L}(\theta) = (r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta))^2, \quad (2)$$

por descenso de gradiente. Entrenar así, sin más, es inestable. DQN lo estabiliza con dos mecanismos de la clase 5.1:

- **Reproducción de Experiencias** (experience replay): guarda las transiciones y entrena con mini-lotes al azar, para romper la correlación entre pasos seguidos.
- **Objetivos-Q Fijos** (red objetivo  $\theta^-$ ): una copia congelada de la red calcula el objetivo, y se sincroniza cada cierto número de pasos. Así el agente no persigue un blanco que se mueve con cada update.

### III-C. Double DQN

El operador máx del objetivo tiende a sobreestimar los valores, porque siempre elige el ruido positivo. Double DQN [3] separa quién elige la acción de quién la evalúa:

$$y = r + \gamma Q(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-), \quad (3)$$

lo que reduce ese sesgo.

### III-D. REINFORCE (gradiente de política)

REINFORCE [6] no aprende valores  $Q$ . Aprende directamente una política estocástica  $\pi(a|s; \theta)$  y ajusta sus parámetros hacia donde crece el retorno:

$$\nabla_{\theta} J \approx \sum_t \nabla_{\theta} \log \pi(a_t|s_t; \theta) G_t, \quad (4)$$

donde  $G_t$  es el retorno descontado desde el paso  $t$ . Su punto débil es la varianza. Para bajarla usamos una línea base simple: dentro de cada episodio normalizamos los retornos, les restamos la media y los dividimos por su desviación estándar [8].

### III-E. Reward shaping

Cuando la recompensa es dispersa, un bono denso ayuda a aprender. El *shaping* potencial [5] promete no cambiar la política óptima:  $F = \gamma \Phi(s') - \Phi(s)$ . En nuestro problema, con  $\gamma < 1$  y rutas largas, esa forma trae un efecto que no queríamos (Sec. VII).

## IV. METODOLOGÍA

### IV-A. El terreno y el MDP

Usamos un recorte de unos  $3 \times 3$  km del modelo digital de elevación (Digital Elevation Model, DEM) Copernicus GLO-30, bajado de AWS Open Data. La Fig. 1 muestra la elevación y la pendiente. El MDP es:

- **Estados:** el agente está en una celda de la malla  $108 \times 108$ .
- **Acciones:** arriba, abajo, izquierda, derecha.
- **Transición:** determinística, u opcionalmente estocástica (el resbalón de la Sec. VI-D).
- **Recompensa:** un costo por paso que crece con la pendiente, un castigo si choca contra un obstáculo, y un premio al llegar a la meta.

### IV-B. Observación local

El agente no ve el mapa completo. Ve un parche de  $9 \times 9$  celdas de pendiente centrado en él (81 valores) más el vector ( $\Delta$ fila,  $\Delta$ columna) hacia la meta, en total 83 números (Fig. 2). Ese tamaño no depende del mapa. Y como terrenos parecidos dan observaciones parecidas, la red puede generalizar. Una tabla no puede: necesitaría una entrada por cada par (posición, meta), del orden de  $10^8$  valores.

### IV-C. Meta fija con inicio aleatorio

La meta es siempre la misma celda; el inicio cambia en cada episodio. Con una sola meta, la función de valor es un campo suave y el entrenamiento se mantiene estable. El inicio aleatorio hace el problema no trivial: el agente aprende a llegar desde cualquier celda, no una ruta memorizada.

### IV-D. Reward shaping de progreso

Con la meta lejos en un mapa grande, el agente casi nunca la encuentra por azar y el premio nunca aparece. Le damos un bono por acercarse. Primero probamos el *shaping* potencial  $\Phi(s) = -k \cdot d(s, \text{meta})$ , y falló (Sec. VII). Lo cambiamos por **shaping de progreso**,  $r += k [d(s, \text{meta}) - d(s', \text{meta})]$ , que suma cero en cualquier ciclo y no crea la trampa del potencial.

### IV-E. Arquitectura e hiperparámetros

La red (tanto la de  $Q$  como la de política) es un perceptrón de  $83 \rightarrow 128 \rightarrow 128 \rightarrow 4$  con activación ReLU, unos 27 mil parámetros. Usamos descuento  $\gamma = 0,99$ , tasa de aprendizaje  $\alpha = 10^{-3}$  con Adam, lotes de 64 transiciones, un búfer de  $10^5$ , pérdida de Huber y recorte de gradiente a norma 10. La red objetivo  $\theta^-$  se sincroniza cada 1000 *updates*. La exploración es  $\epsilon$ -greedy con decaimiento lineal de 1,0 a 0,05 sobre los primeros  $1,5 \times 10^5$  pasos. Cada experimento corre 1500 episodios de a lo más 500 pasos. Un *curriculum*

Tabla I  
ORGANIZACIÓN DEL CÓDIGO

| Archivo        | Contenido                             |
|----------------|---------------------------------------|
| entorno_drl.py | MDP: terreno, observación, recompensa |
| dqn.py         | Red $Q$ , búfer, agente DQN           |
| train.py       | Entrenamiento y test de DQN           |
| reinforce.py   | Red de política, agente REINFORCE     |
| train_pg.py    | Entrenamiento por episodios           |
| baselines.py   | Dijkstra y brecha vs el óptimo        |
| datos.py       | Descarga y recorte del DEM            |
| viz.py         | Figuras                               |
| 01_meta_fija   | DQN principal, rutas, ablaciones      |
| 02_estocastico | Transición estocástica (resbalón)     |
| 03_double_dqn  | Double DQN y meta aleatoria           |
| 04_reinforce   | REINFORCE vs DQN                      |

hace que los inicios arranquen a lo sumo a 20 celdas de la meta y se alejen hasta cubrir el mapa en la primera mitad del entrenamiento. Fijamos la semilla en todos los experimentos para que sean reproducibles. La Tabla II de la Sec. V lista los valores completos.

## V. IMPLEMENTACIÓN

### V-A. Librerías y organización del código

Todo está escrito en Python 3. Usamos **PyTorch** para las redes, el optimizador y el cálculo de gradientes; **NumPy** para el terreno, las máscaras de obstáculos y los generadores aleatorios; **rasterio** para leer el modelo digital de elevación en formato GeoTIFF y recortar la ventana del Misti; y **Matplotlib** para las figuras. Dijkstra lo implementamos a mano con la cola de prioridad *heapq* de la biblioteca estándar, para no depender de un paquete de grafos. No usamos ninguna librería de RL (Gymnasium, Stable-Baselines): el entorno, el búfer, el agente y el bucle de entrenamiento son código propio, lo que nos permitió apagar componentes individuales para las ablaciones.

El código está partido en módulos reutilizables más un notebook por experimento (Tabla I). El entorno y los algoritmos viven en un solo lugar; los notebooks solo los configuran y grafican. Así los cuatro experimentos comparten exactamente el mismo entorno y el mismo procedimiento de test.

### V-B. Procedimiento paso a paso

El flujo completo, desde el dato crudo hasta los números del informe, tiene seis etapas:

1. **Datos.** Se descarga el mosaico Copernicus DSM GLO-30 desde AWS Open Data y se recorta la ventana  $(-71,475, -16,375)$  a  $(-71,445, -16,345)$  en coordenadas geográficas, unos  $3 \times 3$  km. El recorte queda en  $108 \times 108$  celdas de 30 m.
2. **Terreno derivado.** Se calcula la pendiente con el gradiente numérico de la elevación. De ahí salen el costo por celda,  $c = 1 + 8 \|\nabla z\|$ , y la máscara de obstáculos ( $\|\nabla z\| > 0,60$ ). Un recorrido en anchura (Breadth-First Search, BFS) se queda con la componente conexa más grande de celdas libres, para que cualquier par (inicio, meta) que se sortee tenga camino garantizado.

**Algoritmo 1** Entrenamiento DQN con curriculum

```

1: Inicializar red  $Q_\theta$ ; copiar  $\theta^- \leftarrow \theta$ 
2: Inicializar búfer  $\mathcal{D}$  de capacidad  $10^5$ ;  $t \leftarrow 0$ 
3: for  $e = 1$  hasta  $E$  do
4:    $d_{\text{máx}} \leftarrow 20 + \min(1, 2e/E) (216 - 20)$   $\triangleright$  curriculum
5:    $s \leftarrow \text{RESET}(\text{inicio aleatorio con } d \leq d_{\text{máx}})$ 
6:   repeat
7:      $\varepsilon \leftarrow \max(0,05, 1 - t/150000)$ 
8:      $a \leftarrow \arg \max_{a'} Q_\theta(s, a')$  con prob.  $1 - \varepsilon$ , si no al azar
9:      $(s', r, \text{fin}) \leftarrow \text{STEP}(a)$ 
10:    Guardar  $(s, a, r, s', \text{fin})$  en  $\mathcal{D}$ 
11:    if  $|\mathcal{D}| \geq 2000$  then
12:      Muestrear un lote de 64 transiciones de  $\mathcal{D}$ 
13:       $y \leftarrow r + \gamma (1 - \text{fin}) \max_{a'} Q_{\theta^-}(s', a')$ 
14:       $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{\text{Huber}}(Q_\theta(s, a), y)$ 
15:      Recortar  $\|\nabla_\theta\| \leq 10$ 
16:      Cada 1000 updates:  $\theta^- \leftarrow \theta$ 
17:    end if
18:     $s \leftarrow s'$ ;  $t \leftarrow t + 1$ 
19:  until fin o 500 pasos
20:  Cada 100 episodios: evaluar con  $\varepsilon = 0$  sobre los pares de
  test
21: end for

```

3. **Observación.** En cada paso se extrae el parche  $9 \times 9$  de pendiente normalizada centrado en el agente, con relleno de valor 1,0 fuera del mapa (el borde se ve como pared), y se le concatena el vector a la meta dividido por el tamaño del mapa. Son 83 números en  $[-1, 1]$ .
4. **Entrenamiento.** Bucle del Algoritmo 1 para los métodos de valor. Para REINFORCE se junta el episodio completo y se hace un solo update de gradiente de política al final.
5. **Evaluación.** Cada 100 episodios (200 en las ablaciones y en el experimento de divergencia) se corre el test greedy con  $\varepsilon = 0$  sobre 20 pares de test fijos.
6. **Comparación.** Para los pares donde el agente llegó, se calcula la ruta óptima con Dijkstra sobre el mapa completo y se reporta la brecha relativa de costo,  $100 (c_{\text{agente}}/c_{\text{Dijkstra}} - 1)$ .

Las tres banderas de ablación cambian una sola línea del Algoritmo 1. Sin *replay*, la línea 12 se salta y el update usa solo la transición recién vivida. Sin red objetivo, la línea 13 evalúa con  $\theta$  en lugar de  $\theta^-$ . En Double DQN, la línea 13 se reemplaza por el objetivo de la ec. (3).

**V-C. Hiperparámetros**

La Tabla II lista todos los valores. Son los mismos en los cuatro experimentos, salvo el número de episodios y las banderas de ablación.

**V-D. Reproducibilidad**

Fijamos la semilla 42 en NumPy, PyTorch y el generador del entorno al inicio de cada experimento, y el agente recibe además su propia semilla. Los 20 pares de test se sortean con una semilla distinta, 12345, y se generan antes de entrenar: por construcción, los inicios del test no dependen de nada que el agente haya visto durante el entrenamiento. La comparación entre configuraciones (DQN completo, sin *replay*, sin  $\theta^-$ ,

Tabla II  
HIPERPARÁMETROS DEL ENTORNO Y DE LOS AGENTES

| Parámetro                     | Valor                                                     |
|-------------------------------|-----------------------------------------------------------|
| <i>Entorno</i>                |                                                           |
| malla / celda                 | $108 \times 108 / 30$ m                                   |
| peso de pendiente             | 8,0                                                       |
| umbral de obstáculo           | 0,60                                                      |
| lado del parche               | 9 (observación de 83)                                     |
| pasos máximos por episodio    | 500                                                       |
| distancia mínima inicio-meta  | 15 celdas                                                 |
| recompensa de paso            | $-c/10$                                                   |
| choque / meta                 | $-5 / +10$                                                |
| $k$ de <i>shaping</i>         | 0,2                                                       |
| prob. de resbalón             | 0 (o 0,25)                                                |
| <i>Red (valor y política)</i> |                                                           |
| arquitectura                  | $83 \rightarrow 128 \rightarrow 128 \rightarrow 4$ , ReLU |
| parámetros                    | $\approx 27$ mil                                          |
| <i>DQN</i>                    |                                                           |
| descuento $\gamma$            | 0,99                                                      |
| tasa de aprendizaje $\alpha$  | $10^{-3}$ (Adam)                                          |
| tamaño de lote                | 64                                                        |
| capacidad del búfer           | $10^5$                                                    |
| calentamiento                 | 2000 transiciones                                         |
| sincronización $\theta^-$     | cada 1000 <i>updates</i>                                  |
| pérdida / recorte             | Huber / $\ \nabla\  \leq 10$                              |
| $\varepsilon$                 | $1,0 \rightarrow 0,05$ lineal                             |
| horizonte de $\varepsilon$    | $1,5 \times 10^5$ pasos                                   |
| <i>REINFORCE</i>              |                                                           |
| $\gamma / \alpha$             | $0,99 / 10^{-3}$ (Adam)                                   |
| línea base                    | normalización de $G_t$                                    |
| <i>updates</i>                | uno por episodio                                          |
| <i>Común</i>                  |                                                           |
| episodios                     | 1500 (1600 en divergencia)                                |
| curriculum $d_{\text{máx}}$   | $20 \rightarrow 216$ (1. <sup>a</sup> mitad)              |
| pares de test                 | 20, con $\varepsilon = 0$                                 |

Double DQN) usa el mismo entorno recién construido y la misma semilla, de modo que la única diferencia entre corridas es la bandera que se apaga.

La red tiene unos 27 mil parámetros, así que el entrenamiento corre en CPU sin problemas; los notebooks detectan una GPU si está disponible y la usan. Un entrenamiento de 1500 episodios toma unos 12 minutos. El script `gen_figuras_informe.py` reproduce el conjunto completo de figuras en una sola corrida. El DEM no se versiona: `datos.py` lo descarga del bucket público de Copernicus y aplica el mismo recorte, así que el terreno es idéntico en cualquier máquina.

**VI. EXPERIMENTOS Y RESULTADOS****VI-A. Configuración**

El test mide dos cosas de la política greedy: cuántas veces llega a la meta y en cuántos pasos, sobre 20 inicios fijos sorteados con una semilla distinta a la del entrenamiento. Es decir, inicios que el agente nunca vio. Además comparamos el costo de la ruta del agente contra el óptimo exacto de Dijkstra.

**VI-B. DQN con meta fija**

La Fig. 3 muestra la curva de aprendizaje. El agente llega al **100 % de éxito** (20 de 20 inicios) y se queda estable. La Fig. 4

Tabla III  
ABLACIONES: TASA DE ÉXITO EN EL TEST

| Configuración                   | Éxito |
|---------------------------------|-------|
| DQN completo                    | 100 % |
| sin <i>replay</i>               | 0 %   |
| sin red objetivo ( $\theta^-$ ) | 0 %   |
| Double DQN                      | 100 % |

pone cuatro de sus rutas al lado de las de Dijkstra. Sobre los 20 inicios, la brecha media de costo es de solo **3,3 %**, aunque el agente nunca vio el mapa completo, solo parches locales.

#### VI-C. Ablaciones

Repetimos el estudio de ablación de Liu & Zou [4] en nuestro terreno: apagamos cada mecanismo de DQN y medimos qué pasa (Tabla III, Fig. 5). El resultado es claro. Sin la red objetivo ( $\theta^-$ ) el agente no aprende nada (0 %), y sin la Reproducción de Experiencias tampoco (0 %). Los dos componentes son necesarios. Double DQN queda a la par del DQN completo (100 %).

#### VI-D. Transición estocástica

Volvemos el mundo estocástico con un resbalón: con una probabilidad que crece con la pendiente, la acción del agente se cambia por una al azar. La Fig. 6 compara terreno determinista y estocástico. Los dos llegan al 100 % de éxito; el estocástico solo necesita unos pasos más (102 frente a 96). La política aguanta el azar, algo que Dijkstra no modela porque supone que el mundo es perfecto.

#### VI-E. Valor vs política: DQN y REINFORCE

La Fig. 7 compara DQN (basado en valor) con REINFORCE (basado en política) en el mismo terreno. Los dos llegan alto: DQN 100 % y REINFORCE 95 %. La diferencia está en cómo. DQN converge suave y estable, gracias a la Reproducción de Experiencias y la red objetivo. REINFORCE es más simple pero tiene mucha más varianza: la recompensa por episodio salta de un valor a otro. Es el comportamiento típico de cada familia.

#### VI-F. Divergencia con meta aleatoria

La Fig. 8 muestra qué pasa si la meta también es aleatoria (problema *goal-conditioned*). La magnitud media de los valores  $Q$  se mantiene baja hasta cerca del episodio 1000 y luego **explota**: pasa de  $\sim 1$  a más de 2400. Mientras tanto la tasa de éxito nunca sube de 10 %. El agente no aprende una política que se sostenga, y los valores divergen por la sobreestimación del operador máx. Esta es la razón por la que elegimos meta fija.

### VII. DISCUSIÓN

#### VII-A. La trampa del *shaping* potencial

El *shaping* potencial con  $\gamma < 1$  deja un residuo positivo por estar lejos de la meta. Sobre un ciclo entre dos celdas, la recompensa de *shaping* telescopia a  $k(1 - \gamma)(d_X + d_Y) > 0$ .

A distancias grandes eso significa que oscilar lejos de la meta (ir y volver, *ping-pong*) da recompensa positiva, así que la política greedy se queda atrapada yendo y viniendo entre dos celdas. El *shaping* de progreso quita ese residuo y el problema desaparece. Es un caso donde una técnica correcta en teoría, la invariancia de la política óptima, falla en la práctica por cómo interactúa con el descuento.

#### VII-B. Divergencia y la elección de meta fija

Con metas aleatorias, la sobreestimación del máx se re-alimenta con el *bootstrapping* y los valores  $Q$  se disparan. Double DQN retrasa el efecto, pero en nuestro caso no lo evita. La meta fija da un campo de valor único y estable. No es una limitación accidental, es una decisión de alcance con motivo.

#### VII-C. Limitación: política *single-goal*

Nuestra política generaliza sobre el inicio, pero está entrenada para una meta fija. Si se cambia la meta, hay que reentrenar. Llevarla a metas cualesquiera pide técnicas más robustas contra la divergencia, y lo dejamos como trabajo futuro.

### VIII. CONCLUSIONES Y TRABAJO FUTURO

Construimos un agente de Aprendizaje por Refuerzo Profundo que navega terreno real con observación local e inicio aleatorio, y aproxima la función de valor con una red neuronal. El agente llega a la meta desde inicios que nunca vio, con una ruta cercana al óptimo de Dijkstra, y aguanta las transiciones estocásticas. Comparamos las familias de valor y de política, y reportamos dos problemas de diseño con datos que los respaldan. Para el futuro proponemos tres líneas: políticas condicionadas a la meta (*goal-conditioned*) con Hindsight Experience Replay o replay priorizado; mapas más grandes, como el volcán completo; y métodos actor-critic.

### REFERENCIAS

- [1] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
- [2] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.
- [3] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *AAAI*, 2016.
- [4] R. Liu and J. Zou, "The effects of memory replay in reinforcement learning," in *Allerton Conf.*, IEEE, 2018.
- [5] A. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: theory and application to reward shaping," in *ICML*, 1999.
- [6] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, pp. 229–256, 1992.
- [7] V. Mnih *et al.*, "Asynchronous methods for deep reinforcement learning," in *ICML*, 2016.
- [8] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- [9] T. Schaul, D. Horgan, K. Gregor, and D. Silver, "Universal value function approximators," in *ICML*, 2015.
- [10] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," in *ICLR*, 2016.
- [11] M. Andrychowicz *et al.*, "Hindsight experience replay," in *NeurIPS*, 2017.

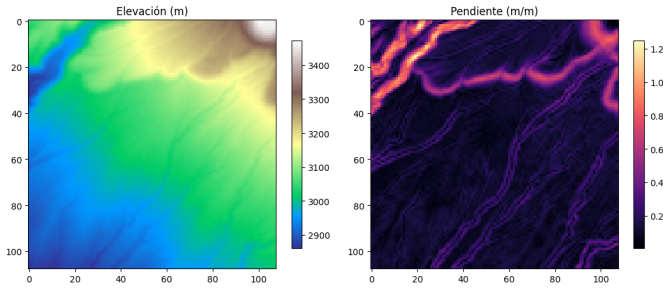


Figura 1. Terreno real (faldas del Misti): elevación (izq.) y pendiente (der.), malla  $108 \times 108$  de 30 m.

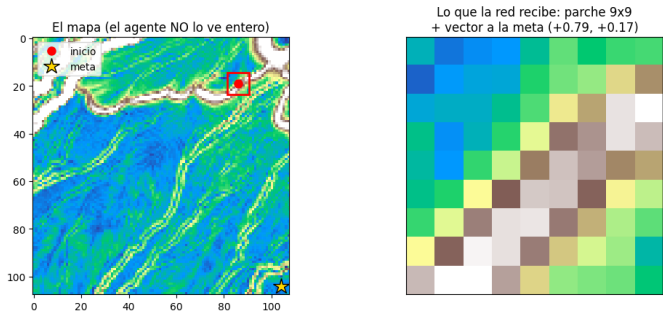


Figura 2. Observación local: el mapa completo con la ventana del agente (izq.) y el parche  $9 \times 9$  que recibe la red (der.), más el vector a la meta.

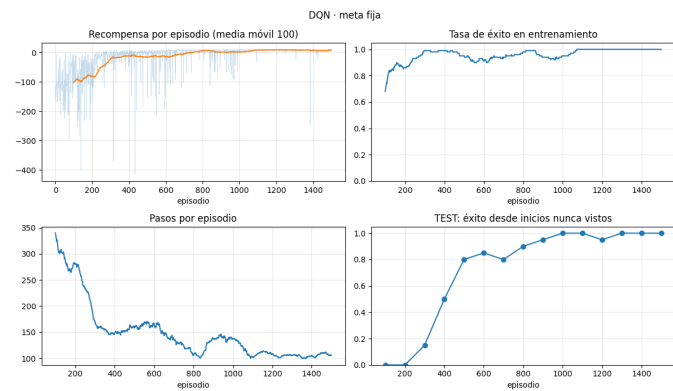


Figura 3. Curva de aprendizaje de DQN con meta fija: recompensa, tasa de éxito de entrenamiento, pasos y test sobre inicios no vistos.

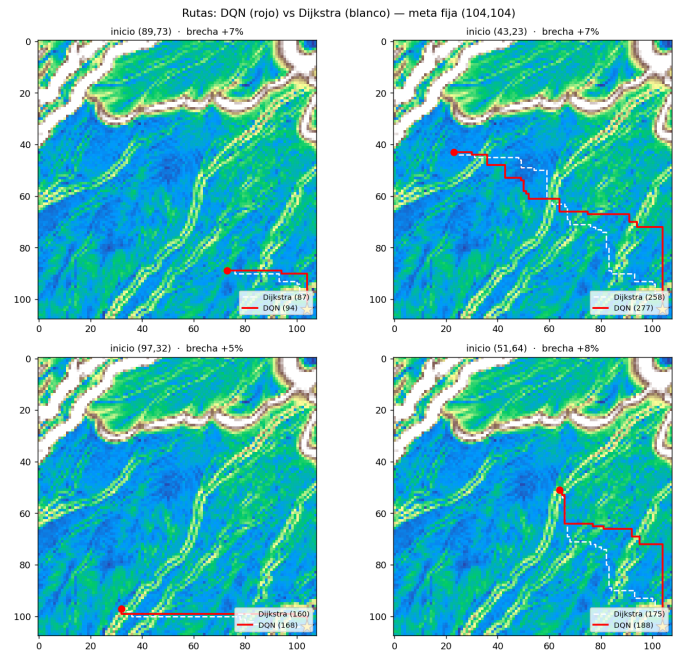


Figura 4. Rutas aprendidas por DQN (rojo) frente al óptimo exacto de Dijkstra (blanco), para cuatro pares inicio-meta de test.

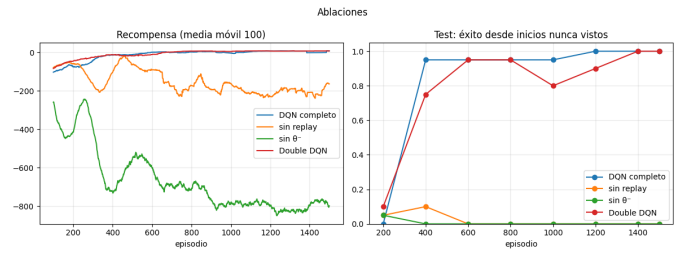


Figura 5. Ablaciones: DQN completo, sin *replay*, sin red objetivo ( $\theta^-$ ) y Double DQN.

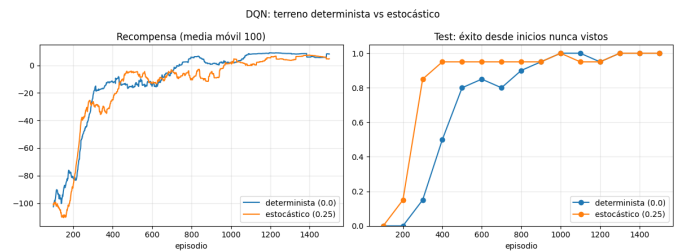


Figura 6. DQN en terreno determinista vs estocástico (resbalón 0,25).

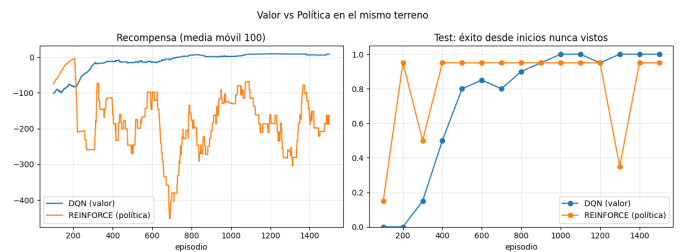


Figura 7. Valor vs política: DQN y REINFORCE en el mismo terreno.

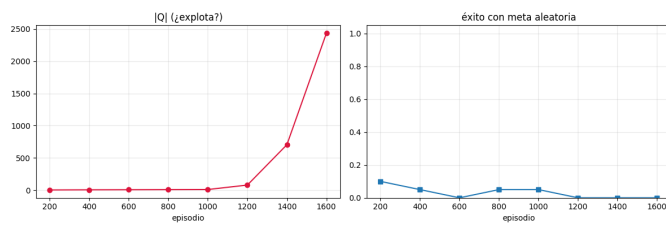


Figura 8. Con meta aleatoria, la magnitud media de  $Q$  (izq.) crece sin control (divergencia) y el éxito (der.) no se sostiene.